

Structure de données : Fonctions de Hachage et Tables de Hachage en Java

Cours d'informatique - CEGEP

9 septembre 2026

1 Introduction et Fonctions de Hachage

Une fonction de hachage transforme une clé de taille arbitraire en un indice entier fixe pour indexer une table de hachage. Voici les principales méthodes :

- **Méthode de la division** : Calcule le reste de la division de la clé par la taille de la table m :

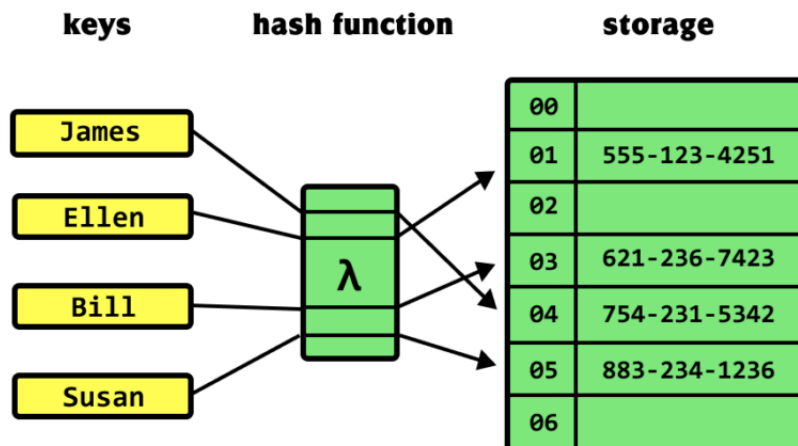
$$h(k) = k \pmod{m}$$

Astuce : m doit idéalement être un nombre premier pour éviter les regroupements périodiques.

- **Méthode de la multiplication** : Multiplie la clé par une constante réelle A ($0 < A < 1$, souvent le nombre d'or ≈ 0.618033), extrait la partie fractionnaire et la multiplie par m :

$$h(k) = \lfloor m \cdot (k \cdot A \pmod{1}) \rfloor$$

- **Hachage d'identité / direct** : Utilisé lorsque les clés sont elles-mêmes des entiers contigus qui correspondent directement aux indices valides du tableau.



2 Tables de Hachage et Collisions

Une **table de hachage** est une structure de données associant des clés à des valeurs. Étant donné que l'espace des clés est souvent plus grand que la taille du tableau, des collisions surviennent (deux clés différentes génèrent le même indice).

- **Chaînage séparé (*Separate Chaining*)** : Chaque case du tableau (un *bucket*) contient une liste chaînée (ou un arbre rouge-noir en Java en cas de forte collision) pour stocker tous les éléments partageant le même haché.
- **Adressage ouvert** : Les collisions sont résolues en cherchant un autre emplacement libre dans le tableau lui-même (sondage linéaire ou quadratique).

3 Implantations en Java : HashMap et HashSet

Java propose des collections robustes basées sur les tables de hachage :

3.1 La classe HashMap<K, V>

Permet de stocker des paires de type clé-valeur avec un temps d'accès moyen en $O(1)$.

- `put(K key, V value)` : Associe la valeur à la clé spécifiée.
- `V get(Object key)` : Retourne la valeur associée à la clé, ou `null` si absente.
- `boolean containsKey(Object key)` : Vérifie si la clé est présente dans la map.
- `V remove(Object key)` : Supprime l'entrée associée à la clé.
- `Set<K> keySet()` : Retourne l'ensemble de toutes les clés.

3.2 La classe HashSet<E>

Représente un ensemble d'éléments uniques, implémenté en interne à l'aide d'un `HashMap` (les éléments du set sont stockés comme les clés, avec une valeur factice partagée).

- `boolean add(E e)` : Ajoute l'élément s'il n'est pas déjà présent.
- `boolean contains(Object o)` : Vérifie si l'élément appartient à l'ensemble ($O(1)$).
- `boolean remove(Object o)` : Retire l'élément de l'ensemble.
- `int size()` : Retourne le nombre d'éléments uniques.
- `void clear()` : Vide entièrement l'ensemble.

4 Exemple d'utilisation en Java

```
import java.util.HashMap;
import java.util.HashSet;

public class ExempleHachage {
    public static void main(String[] args) {
        // Utilisation d'un HashSet pour stocker des elements uniques
        HashSet<String> set = new HashSet<>();
        set.add("Pomme");
        set.add("Banane");

        // Utilisation d'un HashMap pour associer des cles a des valeurs
        HashMap<String, Integer> map = new HashMap<>();
        map.put("Alice", 95);
        map.put("Bob", 82);

        System.out.println("Note de Alice : " + map.get("Alice"));
    }
}
```